

CSP: Cards

È data una collezione di carte, in cui ad ogni carta è associato un intero. La collezione è costituita da $k > 2$ carte per ogni intero da 1 a n , con $n > 0$. Vogliamo disporre in fila le $k \cdot n$ carte in modo che ogni carta contrassegnata dal numero i sia in posizione tale da avere esattamente i carte che la dividono dalla precedente carta contrassegnata da i (se esiste).

1.1 Modellazione

Una formulazione CSP ad alto livello è la seguente.

Variabili Definiamo una variabile per ogni carta:

$$X = \{x_{ij} \mid i \in \{1, \dots, n\}, j \in \{1, \dots, k\}\},$$

Dove i rappresenta l'intero associato alla carta e j rappresenta l'occorrenza della carta con quel numero. Il valore x_{ij} rappresenta la posizione della carta.

Domini

$$\forall x \in X : D(x) = \{1, \dots, n * k\}.$$

Vincoli Ogni carta occupa una posizione unica:

$$C_1 = \text{alldifferent}(\{x_{ij} \mid i \in \{1, \dots, n\}, j \in \{1, \dots, k\}\}).$$

Per ogni carta che non sia la prima occorrenza di un numero, la carta deve essere posizionata a una posizione pari alla precedente occorrenza di quel numero più $i + 1$ (dove i è il numero associato alla carta), in modo da garantire una distanza di i carte tra esse:

$$C_2 = \{\forall i \in \{1, \dots, n\}, j \in \{2, \dots, k\} : x_{ij} = x_{i(j-1)} + (i + 1)\}.$$

$$C = C_1 \wedge C_2$$

1.2 Istanziamento per $(k, n) = (2, 4)$

Si ottiene il CSP (X, D, C) :

$$X = \{x_{1,1}, x_{1,2}, x_{2,1}, x_{2,2}, x_{3,1}, x_{3,2}, x_{4,1}, x_{4,2}\}$$

$$\forall x \in X : \quad D(x) = \{1, \dots, 8\}.$$

$$C_1 = \text{alldifferent}(\{x_{1,1}, \dots, x_{4,2}\})$$

$$C_2 = \left\{ \begin{array}{l} x_{1,2} = x_{1,1} + (1 + 1), \\ x_{2,2} = x_{2,1} + (2 + 1), \\ x_{3,2} = x_{3,1} + (3 + 1), \\ x_{4,2} = x_{4,1} + (4 + 1) \end{array} \right\}$$

$$C = C_1 \wedge C_2$$

1.3 Codifica in MiniZinc

```
1 include "alldifferent.mzn";
2 int: n;
3 int: k;
4 array[1..n, 1..k] of var 1..n*k: x;
5
6 constraint alldifferent(x);
7 constraint forall(i in 1..n, j in 2..k) (
8     x[i,j] = x[i,j-1] + (i+1)
9 );
10
11 solve satisfy;
12
13 output [show(x)];
```

Si può visualizzare la soluzione con la sequenza di carte con questo codice:

```
1 include "alldifferent.mzn";
2 int: n;
3 int: k;
4 array[1..n, 1..k] of var 1..n*k: x;
5
6 constraint alldifferent(x);
7 constraint forall(i in 1..n, j in 2..k) (
8     x[i,j] = x[i,j-1] + (i+1)
9 );
10
11
12 % trasformazione in sequenza
13 array[1..k*n] of var 1..n: sequence;
14 constraint forall(i in 1..n, j in 1..k)(
15     sequence[x[i, j]] = i
16 );
17
18 solve satisfy;
19
20 output [show(sequence)];
```